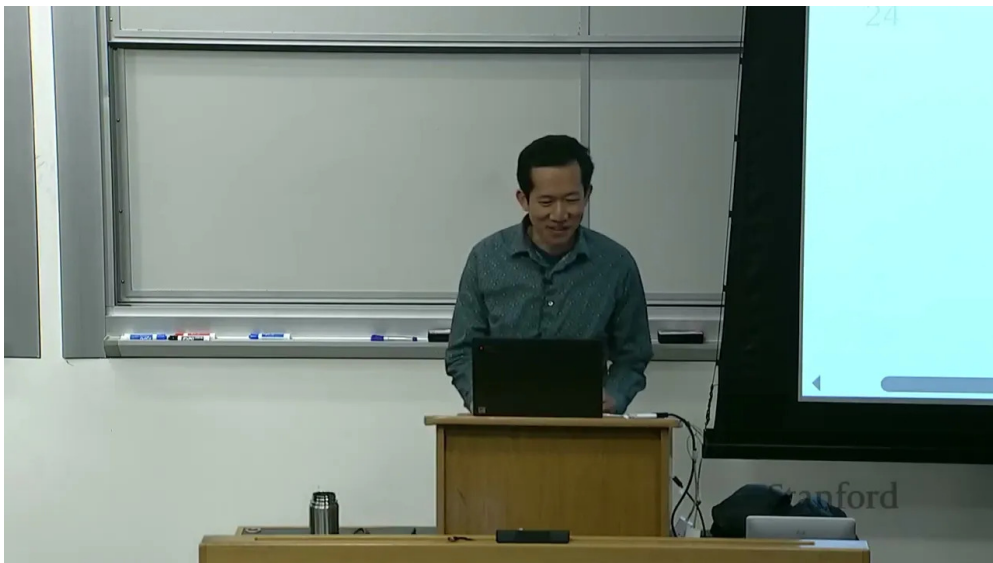


课程笔记

CS336 Lecture 1: 课程导论与整体地图

基于 lecture01.en.srt 与 lecture01-slides.py 整理

2025 年春季



课程主页: <https://stanford-cs336.github.io/spring2025/>

课程阶段: 语言模型 from scratch

本讲主题: 为什么开这门课、课程要学什么、为什么要 “build to understand”

说明: 本目录当前未提供本地 ‘slides-images’, 因此讲义以字幕稿和讲义脚本为主, 图示内容用文字整理。

目录

1 开场：这门课在讲什么	2
2 为什么要开这门课	2
3 工业化时代的语言模型	2
4 这门课能教你什么	3
4.1 经验和实验，有时比解释更诚实	3
5 Bitter Lesson 的正确理解	4
6 课程的历史背景	4
7 这门课的形式：Executable Lecture	4
8 课程整体结构	5
9 课程安排与作业	5
10 为什么特别强调效率	6
11 后续预告：下一讲会讲什么	6
12 本讲小结	6

1 开场：这门课在讲什么

CS336 的第一讲不是直接进入模型细节，而是先回答一个更根本的问题：为什么要在 2025 年再开一门“从零构建语言模型”的课程。讲师一开始就把课程定位得很清楚：这门课不是教你追逐最新的热点技巧，也不是让你依赖一个黑箱 API 直接调用，而是要把语言模型整个栈拆开，重新理解其数据、系统、架构和训练逻辑。

本讲的核心信息

- 课程目标不是“会用模型”，而是“理解模型如何被构建出来”。
- 课程强调完整栈：数据、tokenization、模型结构、训练、系统、规模律、对齐。
- 课程的姿态不是反对抽象，而是警惕抽象泄漏，并在必要时回到机制层面。

讲师给出的课程口号可以概括成一句话：

理解它，必须先构建它。

2 为什么要开这门课

这门课的第一层动机是“研究者正在和底层技术逐渐脱节”。更早的时候，研究者往往自己实现模型、训练模型，至少会下载一个模型并做 fine-tuning；而现在很多工作流已经变成了对一个闭源大模型直接做 prompt。这种抽象当然提高了生产力，也确实释放了大量研究效率，但它也带来了一个问题：抽象是有泄漏的。你未必真的理解系统在底层怎么工作，哪些部分是稳健的，哪些部分只是表面上能跑。

讲师把这个问题说得很直接：如果研究要继续推进，尤其是要做基础研究，就不能只停留在“字符串输入，字符串输出”的黑箱层面。很多真正有价值的问题，需要你把数据、系统、模型这些环节重新拆开，再做协同设计。

这门课的现实背景

- 研究范式已经从“自己训练模型”转向“调用大模型”。
- 这提高了效率，但也让很多人远离了真正的机制理解。
- 课程存在的理由，是让这种底层理解重新变得可学、可练、可验证。

3 工业化时代的语言模型

本讲随后把视角从学术动机转向产业现实：语言模型已经高度工业化了。课程用 GPT-4、xAI 集群、Stargate 之类的例子来说明，前沿模型的训练成本、参数规模和算力投入已经大到普通研究者无法复现。而且最关键的是，这些模型的构建细节并不公开。即使是公开的技术报告，很多时候也只会给出很有限的信息。

这意味着两件事：

- 前沿模型本身已经超出了课堂的直接可达范围。

- 但是这不妨碍我们构建“小模型”来学习机制，只是必须意识到它们未必在规模上完全代表前沿系统。

讲师用两个例子说明“规模改变一切”：

1. 在小模型里，attention 和 MLP 的 FLOPs 可能差不多；但放大到大模型后，MLP 往往成为主导项。
2. 某些能力会在规模上“涌现”，在小规模时几乎看不出来。

这两点共同指向一个结论：只在小规模上优化，不一定能优化到真正重要的地方。

More is different

规模不是简单的线性放大。很多结构、瓶颈和能力只会在更大尺度上显现出来。所以本课程既要教机制，也要教“规模意识”。

4 这门课能教你什么

讲师把课程知识分成三类：mechanics、mindset 和 intuitions。

类别	含义	可迁移性
Mechanics	机制层知识：Transformer 是什么、并行化怎么做、系统如何配合硬件	高，可系统训练
Mindset	规模意识：如何尽可能压榨硬件、如何认真对待 scaling law	高，可明确传递
Intuitions	哪些数据和建模决策真的有效	部分可传递，强依赖尺度与任务

其中 mechanics 和 mindset 是这门课最稳定、最值钱的部分。intuitions 当然也重要，但它更依赖经验、实验和尺度环境，未必能从一个尺度直接外推到另一个尺度。

4.1 经验和实验，有时比解释更诚实

在字幕里，讲师提到很多 Transformer 里的设计，未必都有漂亮的第一性原理解释。比如 SwiGLU 这类非线性，很多时候就是“实验结果确实好”，然后被采用。课程希望传达的不是“我们已经完全解释了所有设计”，而是“实验和规模经常比故事更可靠”。

一句务实的话

如果某个设计选择能在更大规模上稳定提升效果，那么它就值得重视，哪怕它一开始并没有完整的理论解释。

5 Bitter Lesson 的正确理解

讲师专门纠正了对 bitter lesson 的一种常见误读。错误理解是：既然 scale 很重要，那算法就不重要了，只要一直加大资本开模型就行。正确理解恰好相反：真正重要的是“能随着 scale 继续有效的算法”。

换句话说，最终模型效果可以粗略看成

$$\text{accuracy} \approx \text{efficiency} \times \text{resources}.$$

资源当然重要，但效率在大规模下更重要。因为当训练成本已经到数千万、数亿美元时，任何浪费都会被放大。讲师还引用了算法效率的历史改进，强调在很多任务上，算法进步本身可以带来数量级的收益，远远不只是硬件进步。

这门课的底层世界观

- 不是“算法无关”，而是“会随规模成长的算法最重要”。
- 研究的目标不是单纯堆资源，而是提高单位资源的产出。
- 问题可以被重新表述为：在给定 compute 和 data 预算下，能做出多好的模型？

6 课程的历史背景

课程随后快速回顾了语言模型的发展脉络。从 Shannon 用语言模型估计英语熵开始，到 n-gram 在机器翻译和语音识别中的应用，再到神经语言模型、seq2seq、attention、Transformer、MoE、模型并行，最后走向 GPT、BERT、T5、GPT-3、PaLM、Chinchilla，再到近年的开源/开放权重模型。

这段历史的重点不是背名词，而是看清两个趋势：

- 语言模型从“组件”变成了“核心系统”。
- 语言模型从“学术模型”变成了“工业基础设施”。

讲师还强调了开放性层次：

- 闭源模型：只能通过 API 使用。
- 开放权重模型：能看权重和部分训练细节。
- 开源模型：权重、数据和大部分细节都开放。

这对研究很重要，因为“可复现、可检查、可分析”本身就是学习的一部分。

7 这门课的形式：Executable Lecture

课程里有一个很有特色的说法：这是一门 executable lecture。意思不是“讲义里有代码”这么简单，而是：讲义本身就像一个可执行的程序，内容、代码和结构是统一的。

这两个象征性的好处：

- 你可以把学习当作“运行一个系统”，而不是只读静态文档。

- 课程本身就体现了“构建即理解”的方法论。

1 理解系统 -> 实现系统 -> 运行系统 -> 观察系统 -> 继续改进

Listing 1: 课程式讲义的概念性示意

8 课程整体结构

本讲最后给出课程的完整地图。课程以“效率”为主线，把内容分成五个大块：

模块	关心的问题	你会学到什么
Basics	从零搭出一个基本 pipeline	tokenization、Transformer、训练循环
Systems	如何把硬件压榨到极致	kernels、并行训练、推理优化
Scaling laws	如何用小实验预测大规模表现	compute-optimal、超参外推、预算分配
Data	什么数据值得训练、如何处理数据	采集、清洗、过滤、去重、评估
Alignment	如何让 base model 真正可用	SFT、DPO、GRPO、风格和安全

这门课的核心思想是：你永远不是在抽象真空中做研究，而是在资源约束下做选择。因此，课程的每一部分都会反复回到“效率”这个关键词。

9 课程安排与作业

讲师在本讲中也给出了课程 logistics 的信息。课程作业围绕五个方向展开，对应课程的五大模块：

- 基础：实现 tokenizer、Transformer、损失函数、优化器和训练循环。
- 系统：实现 kernel、分布式训练、状态切分和性能分析。
- 规模律：在小规模实验上拟合预测大规模表现。
- 数据：从 Common Crawl 处理出适合训练的数据。
- 对齐：实现监督微调和偏好优化。

课程强调几件事：

- 没有大量脚手架代码，但会提供测试和接口。
- 先在本地验证正确性，再上集群做 benchmark。
- 一些作业有 leaderboard，目标是固定预算下做得更好。
- AI 工具可以用，但会削弱学习效果，需自己权衡。

10 为什么特别强调效率

课程把效率定义成一个总括性的概念：数据、算力、内存、带宽和算法共同决定模型能做到什么。这也解释了为什么课程安排不是按“模型论文”来排，而是按“效率瓶颈”来排。

效率驱动设计

- 数据处理要避免把计算浪费在低质量数据上。
- Tokenization 要在表达能力和计算效率之间折中。
- 模型结构要考虑内存、FLOPs 和硬件友好性。
- 训练要和预算匹配，不是越久越好。
- 对齐和数据选择也会回到资源效率问题。

11 后续预告：下一讲会讲什么

本讲最后把课程推进到下一步：tokenization。课程接下来会从最基础的部分开始，逐步把一个语言模型系统搭起来。下一讲的重点会是 PyTorch 的基本积木、资源账本，以及最小可运行 pipeline 的关键组件。

你应该带走的三句话

1. 这门课不是教你“调用模型”，而是教你“构建模型”。
2. 真正有价值的是能随着规模继续成立的算法和系统。
3. 理解语言模型，必须把数据、系统和模型一起看。

12 本讲小结

第一讲完成了课程定位，也完成了方法论铺垫。它回答了三个问题：

- 为什么要学这门课：因为前沿模型工业化后，基础理解更重要。
- 学什么：机制、规模意识和部分可迁移的直觉。
- 怎么学：从零构建一个完整系统，并持续关注效率。

如果把整门课概括成一句话，那就是：

不要只做会调用模型的人，要做知道模型为什么能工作的那个人。